



CPIT 251 | Software Engineering

Group Project - Final Project Report
Smart System for Redirecting Human Surplus

Prepared by:

Wael Alsulami

Saud Ali Al-Garni

Mazen Althubyani

Ahmed Bahloul

Muhammed Alghamdi

Prof. Awny Sayed

Spring 2026

Contents

1. Abstract.....	3
2. Introduction.....	3
Problem Statement	3
Proposed Solution	3
3. Software Development Process (Scrum Method).....	4
Scrum Process	4
4. Requirements and System Modelling	5
Functional Requirements.....	5
Non-Functional Requirements	6
Use Case Diagram.....	7
Sequence Diagram	8
5. Design.....	9
Software Architecture.....	9
Class Diagram.....	10
6. Input Files	11
7. Implementation	12
Implemented Core Functions.....	12
Application Running Screenshot	13
Output Files Generated.....	13
Output Files Screenshots	14
8. Testing	16
9. Tasks Done by Each Student.....	17
10. Distributed Tasks from GitHub.....	17
11. Conclusion	18
12. Future Work	19

1. Abstract

This project presents a smart volunteer management system called “Smart System for Redirecting Human Surplus.” The system aims to solve the problem of unused human skills and free time by connecting volunteers with suitable volunteer opportunities based on skills, availability, and location. The project was developed using Java and followed the Scrum Agile methodology. The system includes volunteer management, task posting, matching functionality, applications handling, and file management. UML diagrams, software architecture, GitHub collaboration, Trello Scrum board, and JUnit testing were used throughout the project development lifecycle.

2. Introduction

Problem Statement

Many people have free time and useful skills, but they do not know where to use them effectively. Students, graduates, retired people, and part-time workers are often willing to help society but lack a proper channel to participate. At the same time, volunteer organizations need temporary volunteers but face difficulty finding suitable people.

The issue is not the lack of volunteers or opportunities. The real problem is the weak connection between available people and organizations that need help. As a result, a large amount of human effort and time is wasted.

Proposed Solution

The proposed solution is a smart volunteer matching system that connects volunteers with suitable tasks based on:

- Skills
- Available time
- Location

Organizations can post volunteer opportunities, and the system automatically recommends suitable tasks for volunteers. This reduces manual effort and increases community participation.

The system also:

- Stores volunteer data
- Stores task information
- Handles volunteer applications
- Generates output files
- Supports task matching functionality

3. Software Development Process (Scrum Method)

This project followed the Scrum Agile methodology.

The team used:

- Trello for sprint management
- GitHub for collaboration and task distribution
- Scrum workflow for development

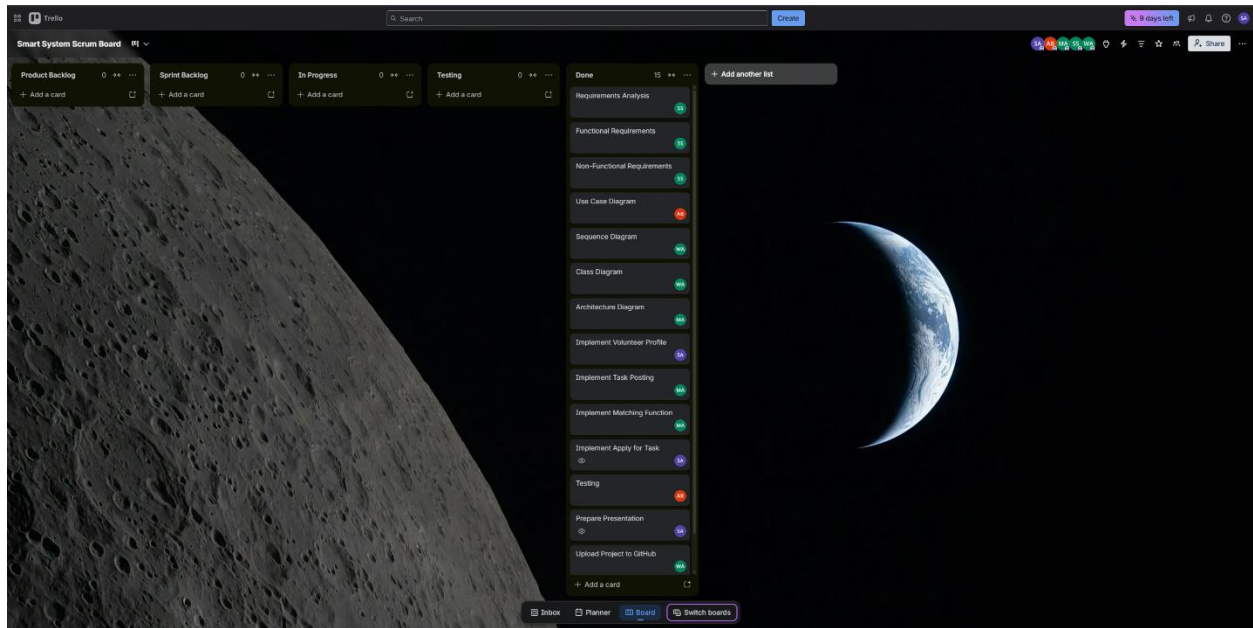
Scrum Process

The project development included:

1. Product Backlog creation
2. Sprint planning
3. Task implementation
4. Testing
5. Final review

The Scrum board included:

- Product Backlog
- Sprint Backlog
- In Progress
- Testing
- Done



4. Requirements and System Modelling

Functional Requirements

Volunteer Functions

- Create volunteer profile
- Update skills and availability
- Browse volunteer tasks
- View recommended tasks
- Apply for tasks
- Cancel applications
- View volunteer hours

Organization Functions

- Post volunteer opportunities
- Edit tasks
- Accept or reject volunteers
- Review applications

System Functions

- Match volunteers with tasks
- Prevent applications for full tasks
- Store application records

Admin Functions

- Approve organizations
- Handle complaints
- Suspend accounts

Non-Functional Requirements

Security

- Role-based access control
- Data encryption
- Security logging

Performance

- Recommendations within 2 seconds
- Support concurrent users

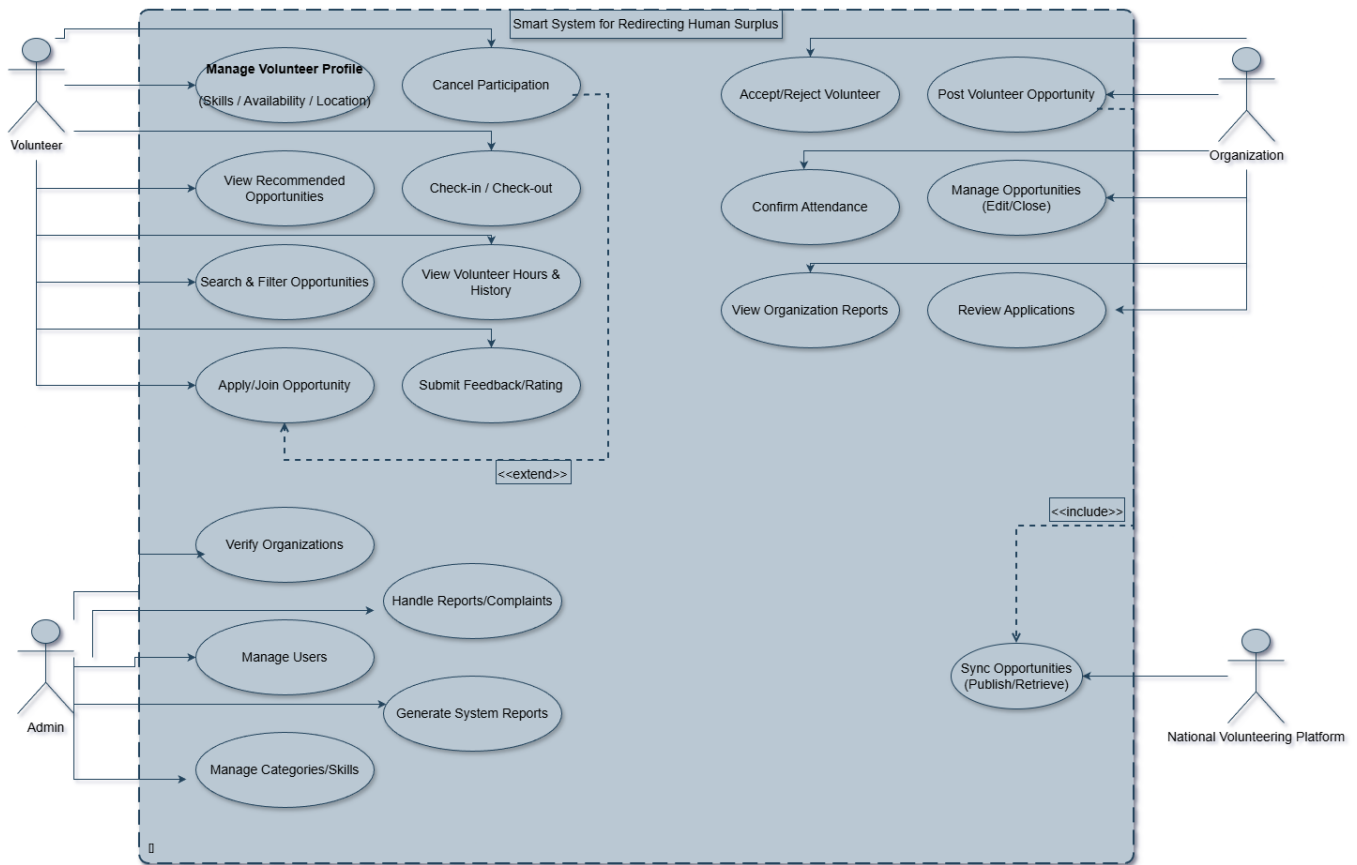
Usability

- Arabic interface support
- Mobile and web support

Maintainability

- Scalable architecture
- Easy modification of matching logic

Use Case Diagram



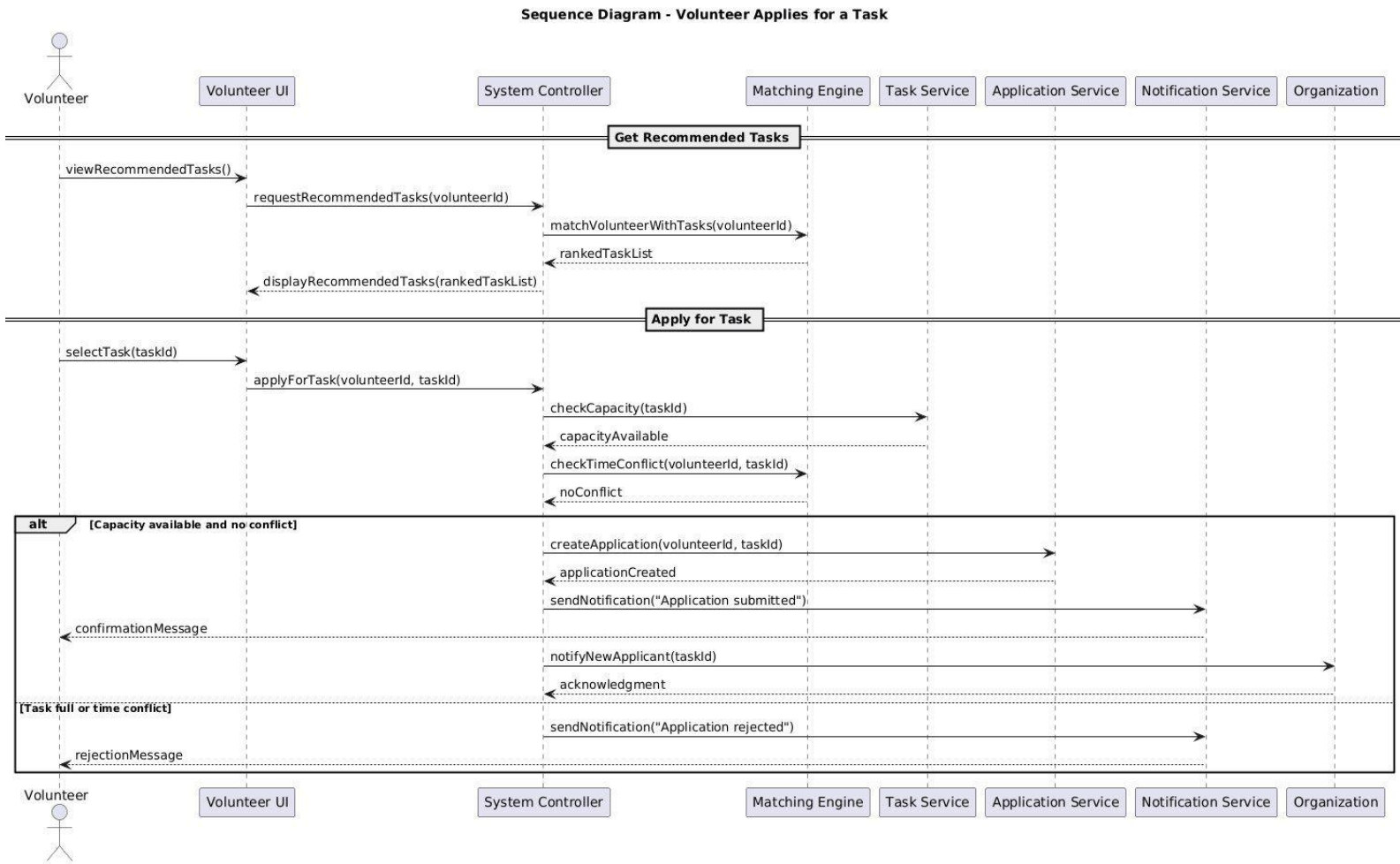
The Use Case Diagram illustrates interactions between:

- Volunteer
- Organization
- Admin
- National Volunteering Platform

Main use cases include:

- Apply for tasks
- Manage opportunities
- Match volunteers
- Confirm attendance

Sequence Diagram

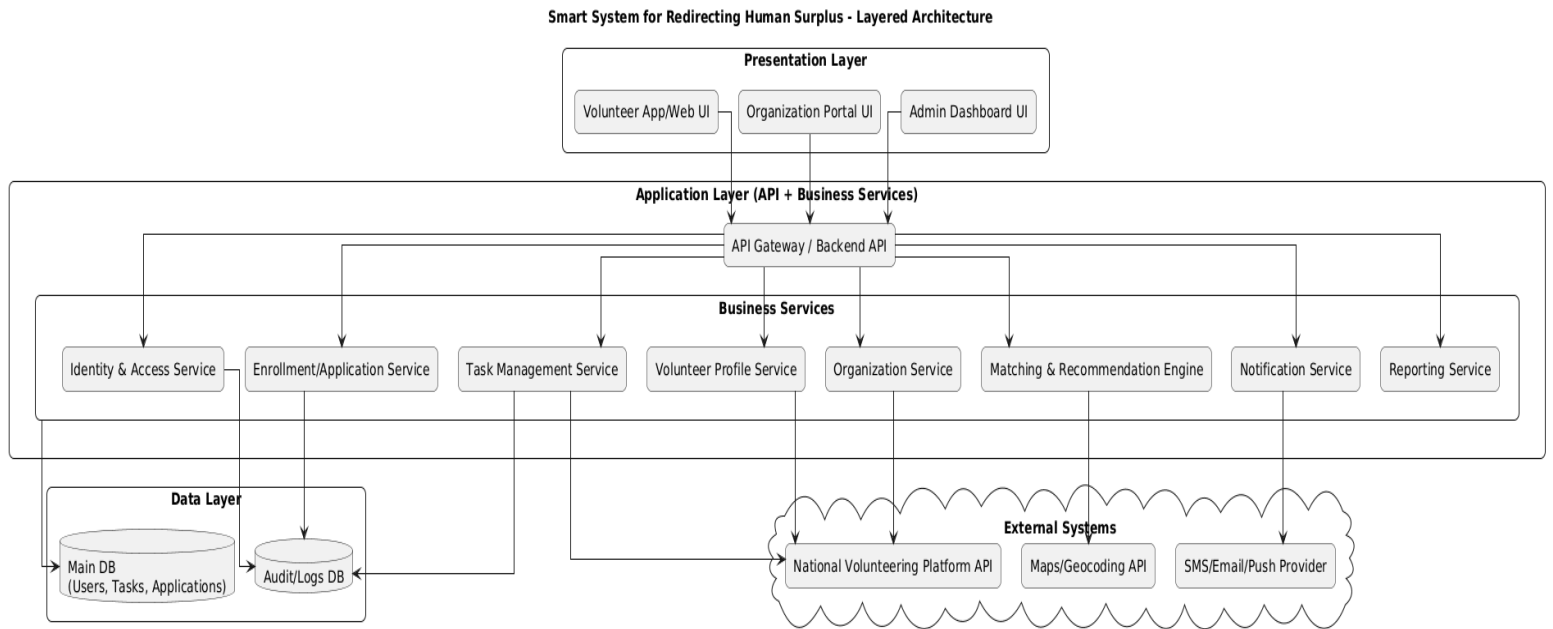


The sequence diagram explains the process of:

1. Viewing recommended tasks
2. Applying for a task
3. Checking capacity
4. Creating applications
5. Sending notifications

5. Design

Software Architecture



The system follows the Layered Architecture Pattern (3-Tier Architecture):

Presentation Layer

- Volunteer UI
- Organization Portal
- Admin Dashboard

Application Layer

- Matching Engine
- Task Management Service
- Notification Service

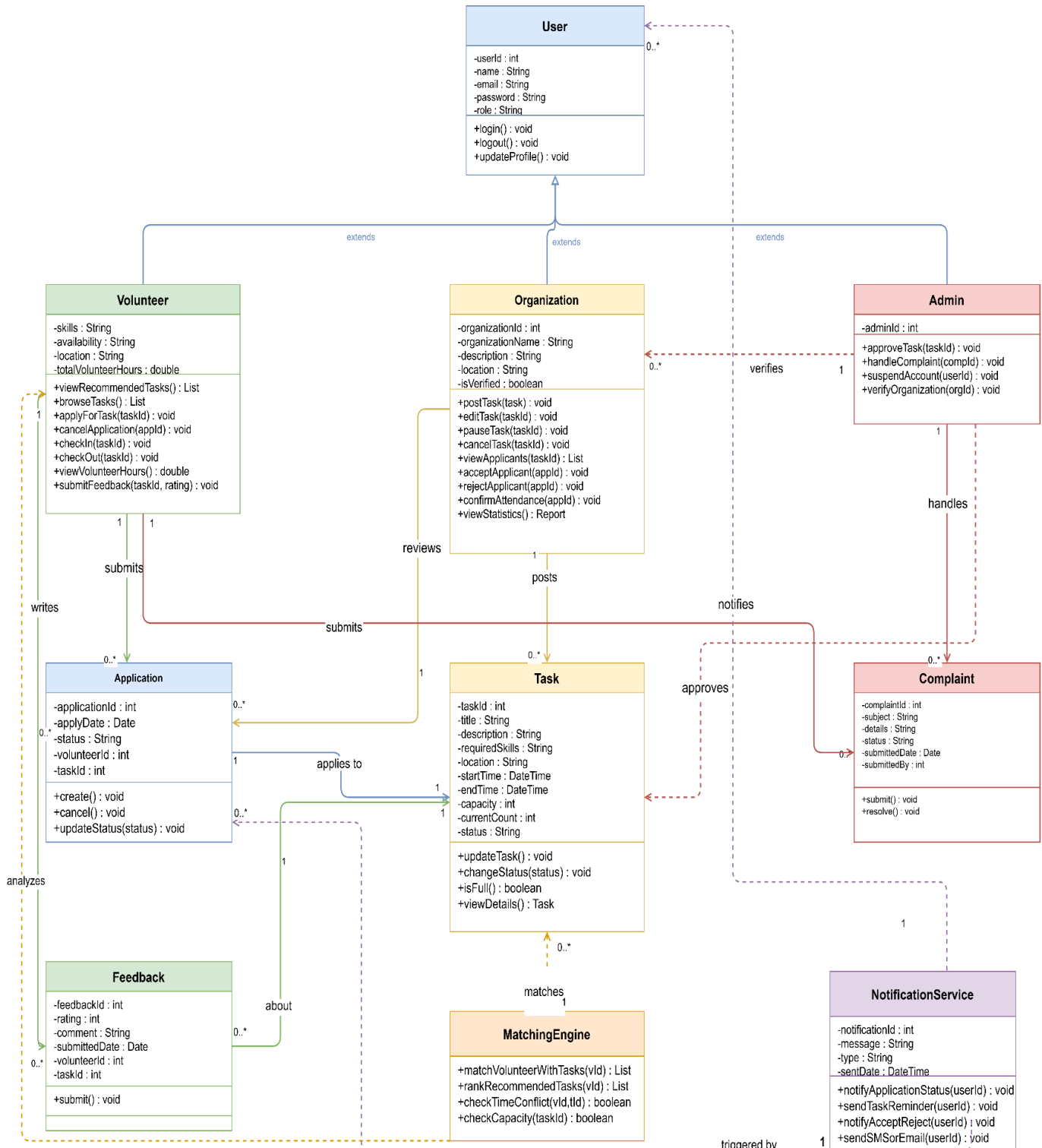
Data Layer

- Volunteer data
- Tasks data
- Applications data

This architecture improves:

- Maintainability
- Scalability
- Separation of concerns

Class Diagram



Main classes include:

- Volunteer
- VolunteerTask
- VolunteerApplication
- MatchingEngine
- DataStore

Relationships between classes handle:

- Applications
- Task matching
- File management

6. Input Files

The system uses input files to load volunteer tasks automatically.

Example input file:

School Tutoring,Teaching,Jeddah,Monday 5PM,3,2

IT Help,IT support,Makkah,Thursday 6PM,2,4

Food Distribution,Organizing,Jeddah,Friday 4PM,5,3

Elderly Support,Communication,Riyadh,Saturday 10AM,4,2

7. Implementation

The project was implemented using Java in NetBeans IDE.

Implemented Core Functions

1. Create Volunteer Profile

Allows volunteers to:

- Enter name
- Skills
- Availability
- Location

2. Post Volunteer Task

Allows organizations to:

- Create tasks
- Define required skills
- Define task capacity

3. Match Volunteer With Tasks

The Matching Engine compares:

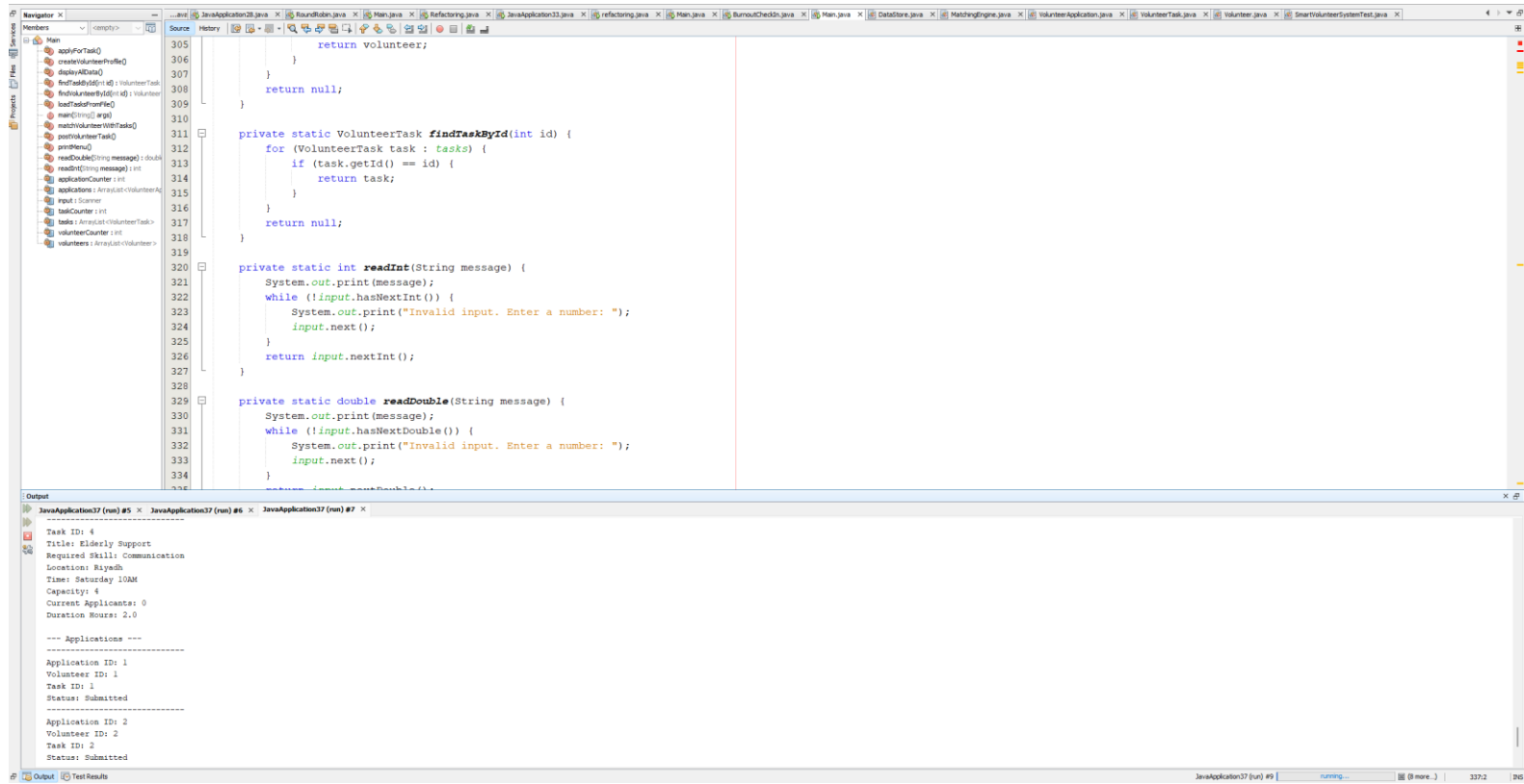
- Skills
- Location
- Availability

4. Apply for Task

The system:

- Checks capacity
- Prevents conflicts
- Creates applications

Application Running Screenshot



Output Files Generated

The system generates:

- volunteers.txt
- tasks.txt
- matches.txt
- applications.txt

Output Files Screenshots

Volunteers Output

```
File Edit View H1  B
Volunteer ID: 1
Name: saud
Skills: teaching
Availability: monday 5pm
Location: jeddah
Total Hours: 2.0
-----
Volunteer ID: 2
Name: mazen
Skills: it support
Availability: thursday 6pm
Location: makkah
Total Hours: 4.0
-----
|
```

Tasks Output

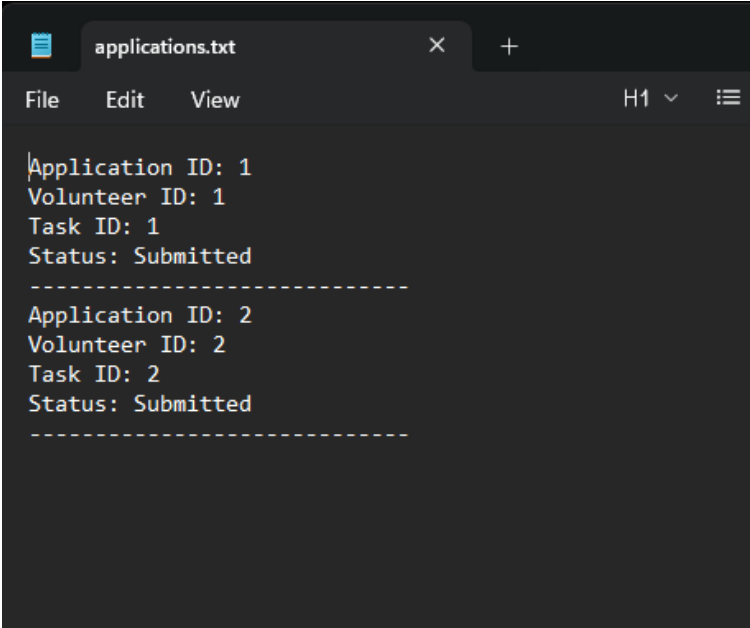
```
File Edit View H1  B I S
Task ID: 1
Title: School Tutoring
Required Skill: Teaching
Location: Jeddah
Time: Monday 5PM
Capacity: 3
Current Applicants: 1
Duration Hours: 2.0
-----
Task ID: 2
Title: IT Help
Required Skill: IT support
Location: Makkah
Time: Thursday 6PM
Capacity: 2
Current Applicants: 1
Duration Hours: 4.0
-----
Task ID: 3
Title: Food Distribution
Required Skill: Organizing
Location: Jeddah
Time: Friday 4PM
Capacity: 5
Current Applicants: 0
Duration Hours: 3.0
-----
Task ID: 4
Title: Elderly Support
Required Skill: Communication
Location: Riyadh
Time: Saturday 10AM
Capacity: 4
Current Applicants: 0
Duration Hours: 2.0
-----
```

Matches Output

```
Matched tasks for volunteer: saud
=====
Task ID: 1
Title: School Tutoring
Required Skill: Teaching
Location: Jeddah
Time: Monday 5PM
Capacity: 3
Current Applicants: 0
Duration Hours: 2.0
-----

Matched tasks for volunteer: mazen
=====
Task ID: 2
Title: IT Help
Required Skill: IT support
Location: Makkah
Time: Thursday 6PM
Capacity: 2
Current Applicants: 0
Duration Hours: 4.0
-----
```

Applications Output

A screenshot of a text editor window titled 'applications.txt'. The window has a menu bar with 'File', 'Edit', and 'View' options, and a status bar showing 'H1' and a list icon. The text content is as follows:

```
Application ID: 1
Volunteer ID: 1
Task ID: 1
Status: Submitted
-----
Application ID: 2
Volunteer ID: 2
Task ID: 2
Status: Submitted
-----
```

8. Testing

JUnit testing was performed on the four implemented core functions.

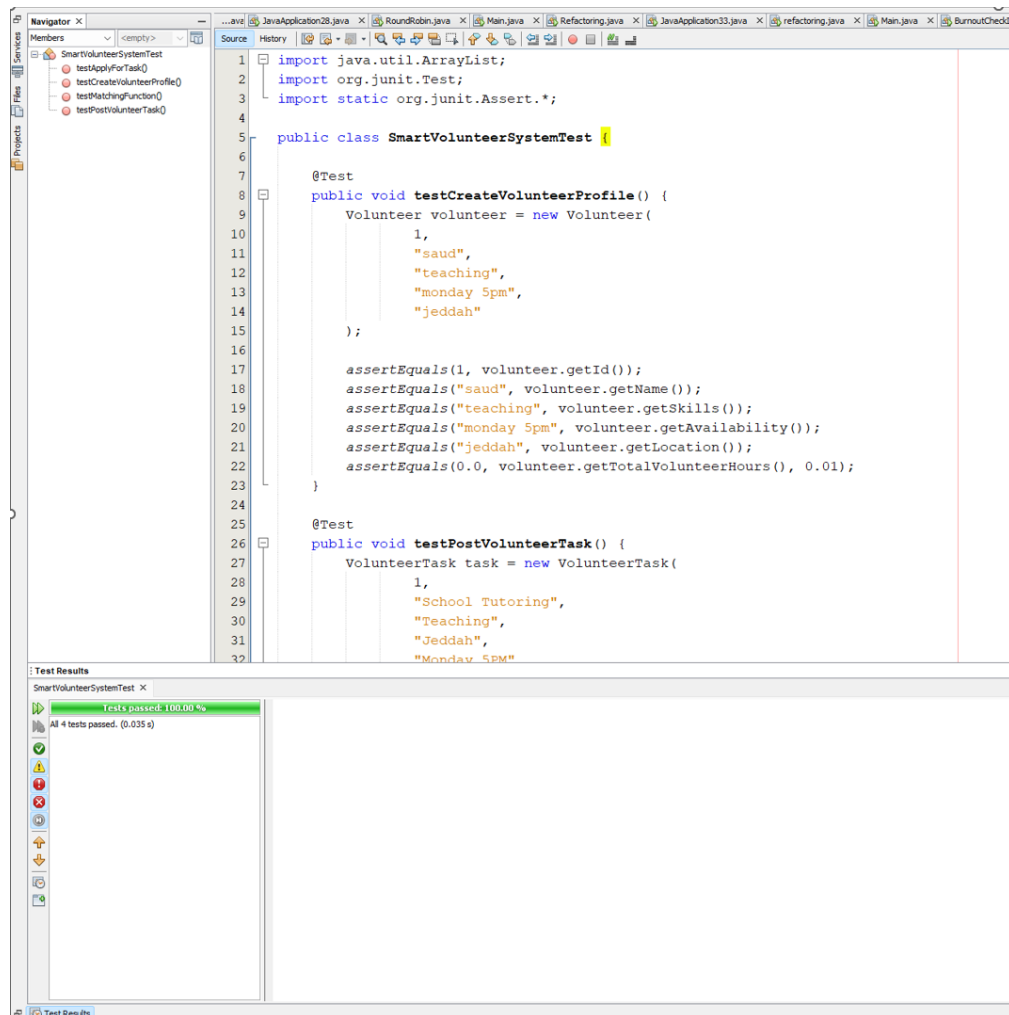
Tested functions:

- testCreateVolunteerProfile()
- testPostVolunteerTask()
- testMatchingFunction()
- testApplyForTask()

The testing process verified:

- Correct volunteer creation
- Correct task creation
- Correct matching logic
- Correct application handling

All tests passed successfully.



9. Tasks Done by Each Student

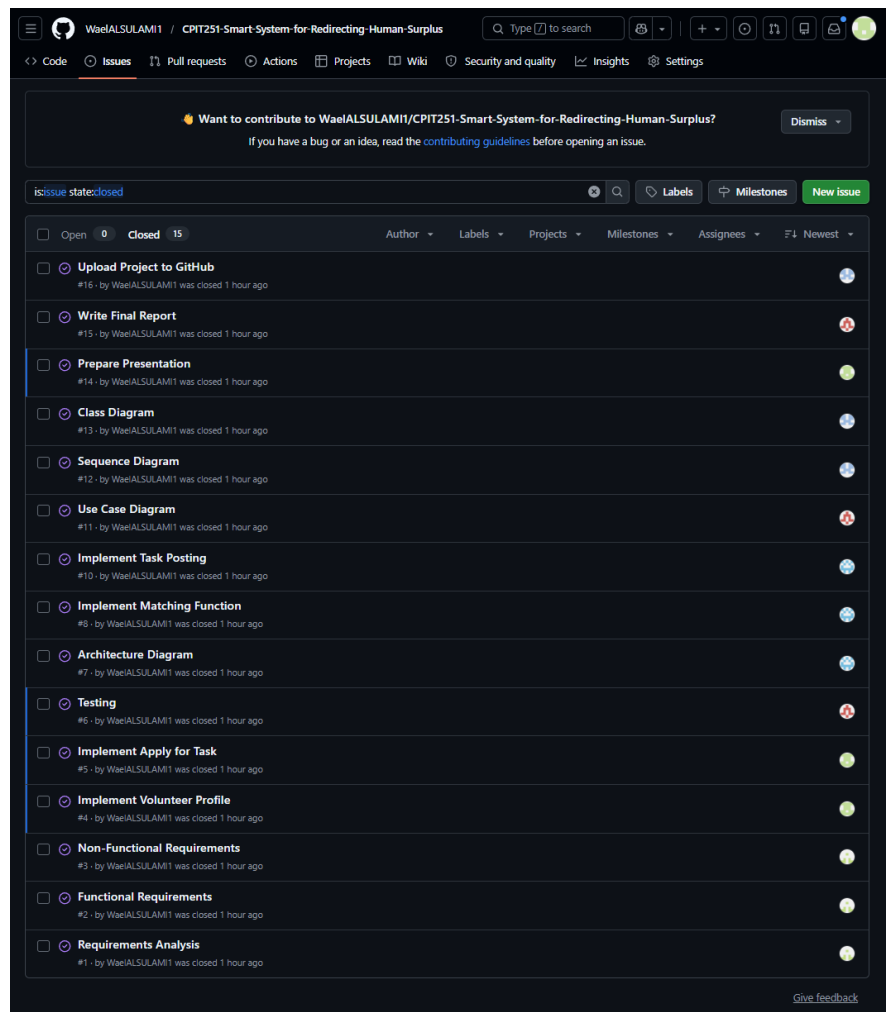
<i>Student</i>	<i>Tasks</i>
<i>Muhammed Alghamdi</i>	Requirements Analysis, Functional Requirements, Non-Functional Requirements
<i>Saud Al-Garni</i>	Implement Volunteer Profile, Implement Apply for Task, Prepare Presentation
<i>Mazen Althubyani</i>	Architecture Diagram, Implement Task Posting, Implement Matching Function
<i>Ahmed Bahlool</i>	Use Case Diagram, Testing, Write Final Report
<i>Wael AlSulami</i>	Sequence Diagram, Class Diagram, Upload Project to GitHub

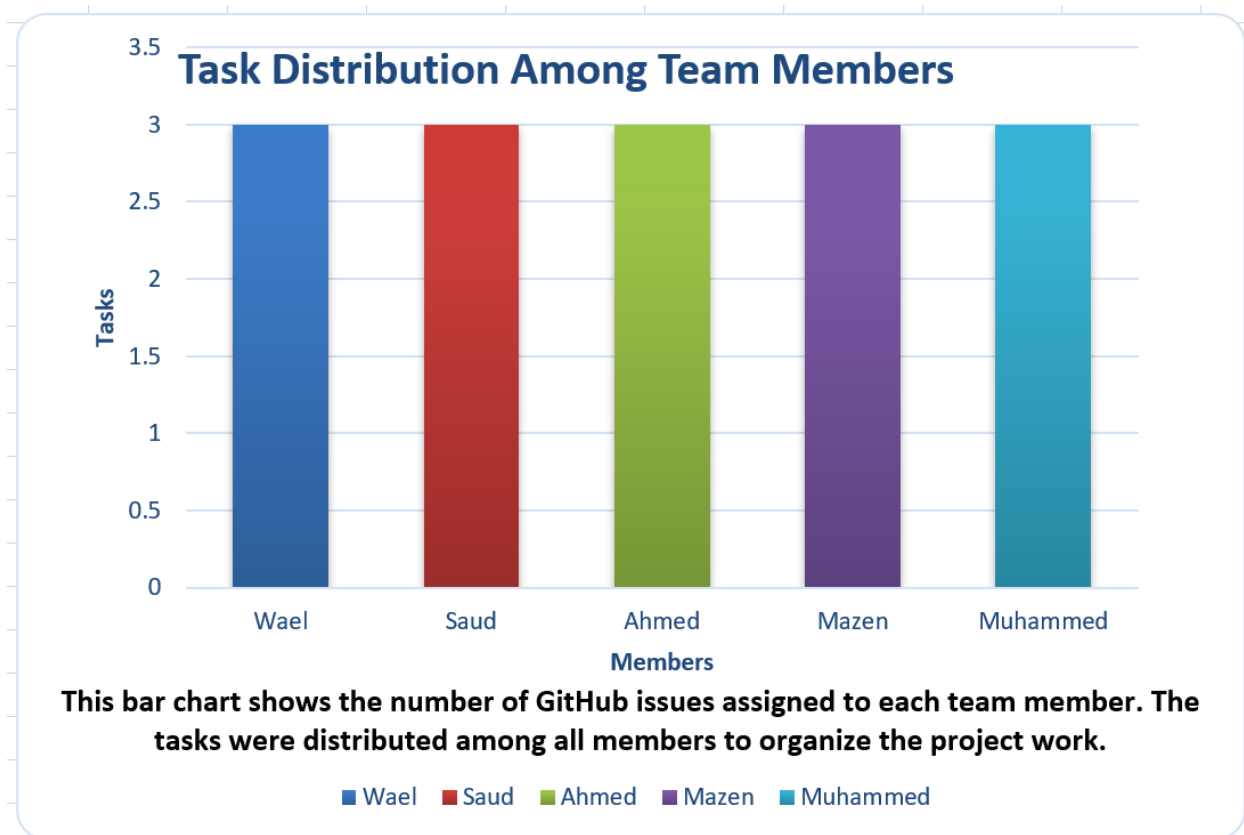
10. Distributed Tasks from GitHub

GitHub Issues were used to distribute tasks among team members.

The team used:

- GitHub Issues
- Assignees
- Scrum workflow





This Figure shows the distribution of project tasks among team members using GitHub Issues. Tasks were distributed equally to ensure collaboration and balanced contribution during the Scrum development process.

11. Conclusion

This project successfully developed a smart volunteer matching system using Java and Scrum Agile methodology. The system solves the problem of unused human skills and free time by connecting volunteers with suitable opportunities based on matching criteria.

The project also demonstrated:

- Object-Oriented Programming
- Software Engineering concepts
- UML modelling
- Scrum workflow
- GitHub collaboration
- Software testing

12. Future Work

Possible future improvements include:

- Developing a graphical user interface (GUI)
- Integrating with real APIs
- Adding a database system
- Adding mobile application support
- Implementing AI-based recommendation algorithms
- Supporting real-time notifications